INTERACTIVE TRAINING · GENERATIVE AI ENGINEERING · DAY 1

# RAG Fundamentals

Retrieval-Augmented Generation: why it exists, how it is built, and your first working retrieval pipeline in Python

🎯 For AI/ML consultants (Python · ML · DL · FastAPI · Docker done) ⏱️ 2–3 hours

🔄 sentence-transformers · NumPy 🔑 No LLM API key needed today

Objectives → Why RAG? → RAG vs fine-tuning → Architecture → Simulation → Chunks · embeddings · search → **Live Python retrieval** → Exercises → Quiz → Day 2

🧪 **READ THIS FIRST — SIMULATIONS VS REAL MODELS**

Every interactive widget in this page runs in your browser with **hand-made, deterministic numbers** so the behaviour is easy to explain. They are labelled ILLUSTRATIVE SIMULATION . No embedding model or LLM runs in this HTML file. The **real** embedding model runs only in the Python code of Module 6, on your own machine.

00 MODULE 0

## Learning Objectives and Session Roadmap

⏱ 5 min

You already know how to train models, serve them with FastAPI and package them with Docker. Today you learn the most common architecture for putting **your organisation's own knowledge** behind a Large Language Model: **Retrieval-Augmented Generation (RAG)**.

### 🎯 By the end of Day 1 you can...

1. Explain what RAG is and why it is useful.
2. Explain the limitations of a standalone LLM.

3 Compare RAG, fine-tuning and prompt engineering.

4 Draw and explain the RAG architecture.

5 Explain ingestion, chunking, embeddings, vector storage and retrieval.

6 Understand how semantic similarity search works.

7 Implement a basic retrieval pipeline in Python.

8 Explain the difference between *retrieving context* and *generating an answer*.

## 🗺️ Session roadmap

Click a phase to jump to it.

📖
**Theory**
**30 min**
Modules 0–2

**Architecture**
**20 min**
Module 3

🧪
**Simulation & concepts**
**35 min**
Modules 4–5

**Live coding**
**35 min**
Module 6

🛠️
**Exercises & check**
**25 min**
Modules 7–9

Core: ~145 minutes. Items marked OPTIONAL add 5–15 minutes and can be skipped if time is short.

🧭 **TODAY'S RUNNING EXAMPLE**

A small set of **company HR and IT policies**. Simple enough to reason about by eye, realistic enough to show every RAG problem: matching different wording, choosing how much context to send, and knowing when the documents simply do not contain the answer.

📍 **SCOPE OF DAY 1**

**In:** concepts, architecture, a working *retrieval* pipeline. **Not yet:** calling an LLM to generate answers, advanced chunking and retrieval tuning, evaluation, production deployment — those come in later sessions.

---

➡️ **TRANSITION TO THE NEXT TOPIC**

*"We know where we are going. First question: if LLMs are so capable, why do we need retrieval at all?"*

01

MODULE 1 · THE PROBLEM

# Why Do We Need RAG?

⏱ 15 min

## What is a Large Language Model?

A Large Language Model (LLM) is a neural network — a Transformer, the architecture you met in Deep Learning — trained on very large amounts of text to **predict the next token**. Repeating that prediction produces fluent answers, summaries, code and explanations.

### 🧠 What an LLM uses to answer

**1. Parametric knowledge** — patterns stored in its weights during training.

**2. The prompt (context window)** — everything sent in the current request: instructions, conversation, and any documents you paste in.

Nothing else. It does not browse your file share or database unless your application fetches that data and puts it into the prompt.

### ⚠️ Limits of pretrained knowledge

**Knowledge cut-off:** it knows nothing after its training data ends.

**No private data:** your company's policies, contracts, tickets and wikis were never in its training set.

**No sources:** knowledge in weights cannot be cited or traced to a document.

**Expensive to update:** changing weights means retraining or fine-tuning.

## How hallucinations happen

An LLM is optimised to produce *plausible* text, not to check facts. When the needed fact is missing, the most plausible continuation may still be a confident, specific — and wrong — answer. Ask about "our annual leave allowance" and a model may answer "20 days" because that number is common in text it has seen, not because it knows your policy.

## Why not paste every document into every prompt?

### 📏 Context limits

Context windows are large but finite. Thousands of policy pages, tickets and manuals will not fit.

### 💸 Cost & latency

You pay for and wait on every input token, for every question — even when 99% of the text is irrelevant.

### 🎯 Focus

Burying one relevant sentence among thousands of irrelevant ones can make it harder for the model to use the right evidence.

💡 **THE RAG IDEA**

First retrieve the few passages relevant to the question, then augment the prompt with them, then let the LLM generate an answer grounded in that evidence.

## The example: one question, three approaches

📄 **Doc 1**
"Employees receive 18 days of annual paid leave."

📄 **Doc 2**
"Remote work is permitted up to three days per week with manager approval."

📄 **Doc 3**
"Employees must submit expense claims within 30 days of the transaction."

❓ **Question**
**"What is the annual paid leave allowance?"**

▶ **Play the three scenarios side by side** ILLUSTRATIVE SIMULATION — NO REAL LLM

### A · Standalone LLM

No company documents in the prompt

Prompt size –

### B · Whole policy pasted in

All relevant policy text in the prompt

Prompt size –

### C · RAG

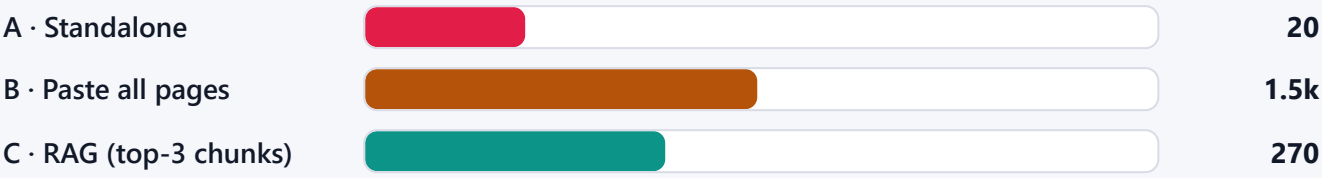
Retrieve the relevant passage first

Prompt size –

Responses are scripted to show typical behaviour. A real standalone model might refuse, ask for clarification, or guess — guessing is the risk shown in A.

## Scaling up: how prompt size grows with the document collection

**Drag to grow the knowledge base** ILLUSTRATIVE NUMBERS




Approximate input tokens per question (log scale; ~500 tokens per page, illustrative). At this size, pasting everything is a reasonable option.

| | A · STANDALONE LLM | B · PASTE EVERYTHING | C · RAG |
|---|---|---|---|
| **Knows your policy?** | **No** | **Yes, if it fits** | **Yes, if retrieval finds it** |
| **Prompt size** | Tiny | Grows with the collection | Small and roughly constant (top-K passages) |
| **Can cite sources?** | **No** | Possible | **Yes — retrieved chunks carry metadata** |
| **Main risk** | Confident guess (hallucination) | Cost, latency, context limit, distraction | Retrieval misses or returns the wrong passage |

⚖️ **BE PRECISE WITH CLIENTS**

RAG **reduces** hallucination risk by giving the model relevant evidence and an instruction to use it. It does **not eliminate** hallucinations: retrieval can return the wrong passage, the documents can be outdated or contradictory, and the model can still misread or ignore the context. Grounding, source citations, abstention rules and evaluation are all needed.

**CHECK 1**

### Why does a standalone LLM not know your company's leave policy?

A. Its context window is too small for any policy

**B. The policy was not in its training data and is not in the prompt**

C. LLMs cannot process numbers

D. It needs to be deployed in Docker first

---

➡️ **TRANSITION TO THE NEXT TOPIC**

*"RAG is one way to give a model knowledge. Fine-tuning and prompt engineering are others — when should you choose which?"*

02

MODULE 2 · CHOOSING THE TOOL

# RAG vs. Fine-Tuning vs. Prompt Engineering

⏱ 10 min

These three techniques solve **different problems** and are often combined. A useful rule: *RAG changes what the model knows for this request; fine-tuning changes how the model behaves; prompting tells the model what to do now.*

**Interactive comparison — click a column title to highlight it**

| DIMENSION | 🔎 RAG | 🏋️ FINE-TUNING | ✍️ PROMPT ENGINEERING |
|---|---|---|---|
| **Primary purpose** | Supply relevant external knowledge at query time | Adapt behaviour, style, format or task skill | Instruct the model with rules, examples and constraints |
| **Model weights change?** | **No** | **Yes (all or some, e.g. LoRA adapters)** | **No** |
| **Retrieves documents at query time?** | **Yes** | **No** | **No (unless combined with RAG)** |
| **Cost & complexity** | Moderate: ingestion pipeline, embeddings, vector store, retrieval tuning | Higher: curated training data, GPU training, evaluation, re-training for changes | Lowest: text changes, fast iteration |
| **Knowledge freshness** | Update by re-indexing documents — minutes | Frozen until the next training run | Only what you write into the prompt |
| **Source citations** | Natural — chunks carry source metadata | Not traceable to documents | Only for text you supplied |
| **Typical use cases** | Q&A over policies, manuals, tickets, contracts, knowledge bases | Consistent output format, domain tone, classification/extraction skill, smaller cheaper model for a narrow task | One-off tasks, role/format instructions, few-shot examples, guardrails |
| **Limitations** | Quality limited by retrieval; poor | Poor way to inject frequently changing facts; risk of | Limited by context size; can't add |

| DIMENSION | | | |
|---|---|---|---|
| | chunks → poor answers; extra latency | overfitting; needs data and MLOps | large private knowledge |

### 🔎 RAG example

Answer employee questions from company policies that HR updates every month.

### 🏋️ Fine-tuning example

Make a model always return insurance claim summaries in a strict 7-field JSON format with company terminology.

### ✍️ Prompting example

"Rewrite this release note for executives in five bullet points" — once.

## 🧭 Decision simulator

**Pick a client requirement — get a reasoned recommendation**

- ○ **Scenario 1.** A company updates its policy documents every week; staff ask questions about them.
- ○ **Scenario 2.** Responses must consistently follow a specialised output format and domain tone.
- ○ **Scenario 3.** A model needs instructions for a one-time task (e.g. summarise this report for executives).
- ○ **Scenario 4.** An application must answer questions from private technical documentation (thousands of pages).

👈 Select a scenario.

🧩 **THEY COMBINE**

Real systems often use all three: **prompt engineering** for instructions ("answer only from the context, cite sources"), **RAG** for knowledge, and sometimes **fine-tuning** for format or domain behaviour.

**CHECK 2**

**A client's product catalogue changes every week and the assistant must answer questions about current products. First choice?**

A. Fine-tune weekly on the catalogue

B. A longer system prompt with a few examples

**C. RAG over the catalogue, re-indexed when it changes**

D. Train a new model from scratch

---

➡️ **TRANSITION TO THE NEXT TOPIC**

*"RAG is the right tool for private, changing knowledge. Let's open the box and look at every component of a RAG system."*

MODULE 3 · THE BIG PICTURE

03

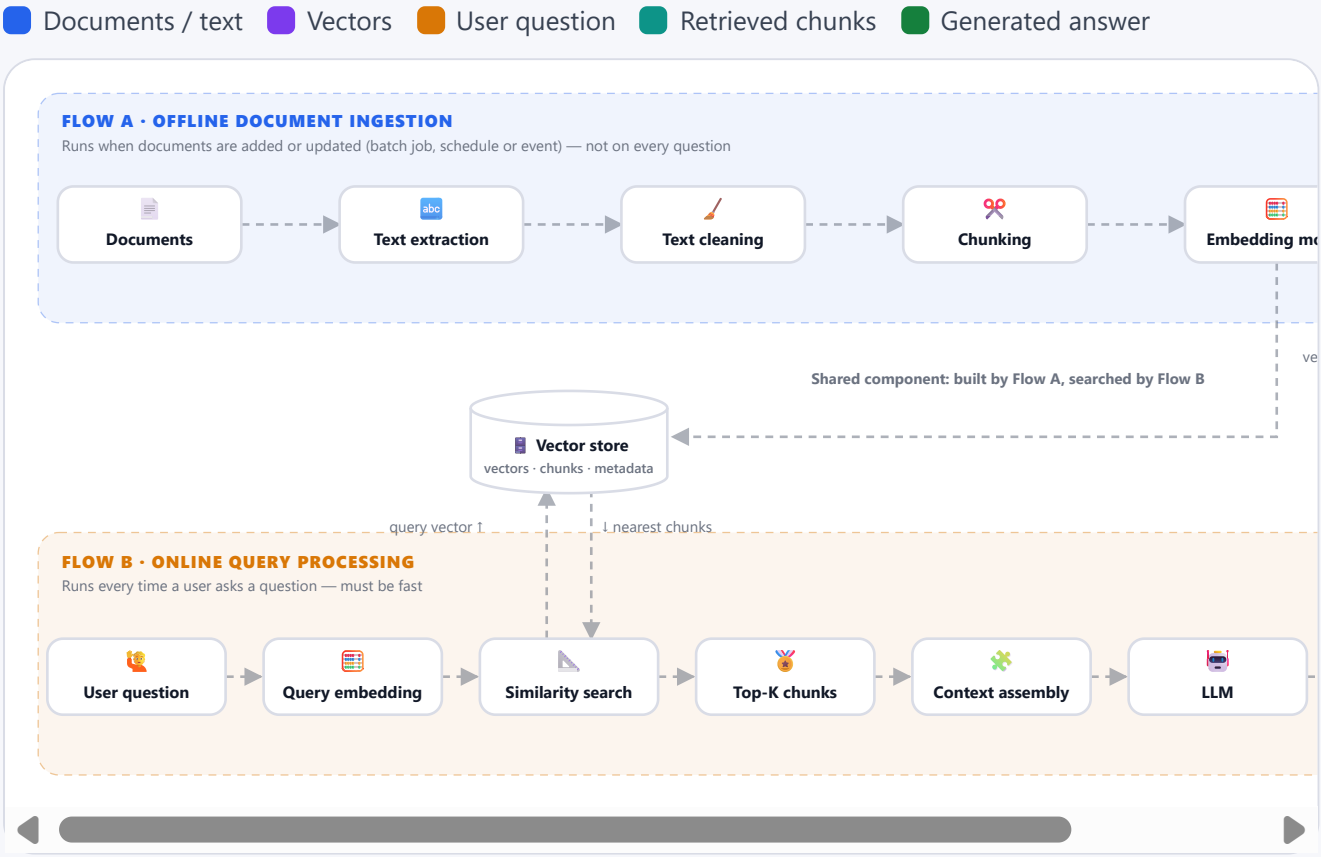
# Complete RAG Architecture

⏱ 20 min

A RAG system is **two pipelines** that meet at the vector store. The **ingestion** pipeline prepares documents ahead of time; the **query** pipeline runs every time a user asks a question.

**Animated RAG architecture — press Start, click any component for details**

CONCEPTUAL ANIMATION




**Ready** Press **Start animation** to send documents through ingestion, then a question through the query pipeline.

**WHAT EXACTLY IS IN A VECTOR STORE?**

Each entry holds a **vector** (the searchable numeric representation) and usually the **chunk text** (or a reference to where it lives) plus **metadata** such as source file, page, department and last-updated date. A vector is *not* a readable copy of the text — you cannot simply read the original sentence back out of the numbers. That is why stores keep the text or a pointer to it: after the search finds the nearest vectors, the application needs the actual words to send to the LLM.

## Every component: what it receives and what it produces

| # | COMPONENT | RECEIVES | PRODUCES |
|---|---|---|---|
| 1 | 📄 **Document source** Ingestion | Files from SharePoint, file shares, wikis, ticket systems, databases, websites. | Raw files: PDF, DOCX, HTML, Markdown, TXT. |
| 2 | 🔤 **Document loader & text extraction** Ingestion | Raw files (binary or markup). | Plain text per document/page, plus basic metadata (file name, page number, title). |
| 3 | 🧹 **Text preprocessing** Ingestion | Raw extracted text. | Clean text: headers/footers removed, hyphenation fixed, whitespace normalised, boilerplate dropped. |
| 4 | ✂️ **Chunking** Ingestion | Clean text of each document. | Chunks: short passages with IDs and metadata (source, page, section). |
| 5 | 🧮 **Embedding model** Ingestion | Chunk texts (in batches). | One fixed-length vector per chunk, e.g. 384 numbers for all-MiniLM-L6-v2. |
| 6 | 🗄️ **Vector database / index** Shared | Vectors + chunk text (or references) + metadata. | A searchable index that returns the nearest vectors to a query vector, with their text and metadata. |
| 7 | 🙋 **User query** Query | A natural-language question from a user or application. | The query text (optionally with user/permission info). |
| 8 | 🧮 **Query embedding** Query | The query text. | A query vector of the same dimension, from the same model used at ingestion. |
| 9 | 📐 **Similarity search** Query | The query vector. | Similarity scores between the query vector and stored chunk vectors (e.g. cosine). |

| | | | |
|---|---|---|---|
| 10 | 🏅 **Top-K retrieval** Query | All scores (ranked). | The K highest-scoring chunks with text, source metadata and scores. |
| 11 | 🧩 **Context assembly** Query | Top-K chunks + the user's question + instructions. | A prompt: instructions, numbered context passages with sources, the question. |
| 12 | 🤖 **LLM generation** Query | The assembled prompt. | Generated natural-language text. |
| 13 | ✅ **Answer & source references** Query | Generated text + metadata of the chunks used. | An answer shown to the user with citations (file, page) — or "insufficient information". |

## 🪜 Pipeline stepper — inspect each stage's input and output

**Step through one document and one question** ILLUSTRATIVE VALUES

🗂️ Ingestion pipeline · stage 1 of 13 📄 **Document source**

An HR handbook is added to the document library.

← INPUT

```
hr_leave_policy.pdf  (binary PDF, 52 pages)
```

→

OUTPUT →

```
File handle + metadata: {source: 'hr_leave_policy.pdf', owner: 'HR', modified: '2026-01-15'}
```

⚡ CHECK 3

### When does the document-embedding step normally run?

A. Every time a user asks a question

B. Only once, ever

C. Inside the LLM

**D. When documents are added or updated, in the ingestion pipeline**

➡️ **TRANSITION TO THE NEXT TOPIC**

*"We've seen the pipeline as boxes and arrows. Now type your own questions into a working (simulated) pipeline and watch retrieval decide what the LLM gets to see."*

04

MODULE 4 · HANDS ON THE PIPELINE

# Interactive RAG Pipeline Simulation

⏱ 15 min

🧪 **HOW THIS SIMULATION WORKS — HONESTLY**

JavaScript cannot run a real embedding model here. Instead, each question is mapped to **six hand-made "semantic features"** (leave, remote work, expenses, IT support, time/schedule, general policy) using a small keyword-and-synonym list. Each document has fixed feature values. The scores are the **cosine similarity of these illustrative vectors** — deterministic and explainable, but much simpler than a real model, which learns hundreds of dimensions from data. The "answer" is a scripted template, not LLM output.

### 📄 Document A

"Employees receive 18 days of annual paid leave."

### 📄 Document B

"Remote work is permitted up to three days per week with manager approval."

### 📄 Document C

"Expense claims must be submitted within 30 days of the transaction."

### 📄 Document D

"The IT helpdesk operates from 9 AM to 6 PM on weekdays."

## Ask a question

ILLUSTRATIVE SIMULATION

Question → Features / vector → Similarity → Rank & Top-K → Context → Answer

## 1 · Question

No question yet.

## 2 · Illustrative query vector

NOT A REAL EMBEDDING

| | | |
|---|---|---|
| Leave / time off | | – |
| Remote work | | – |
| Expenses | | – |
| IT support | | – |
| Time / schedule | | – |
| General policy | | – |

## 3 · Similarity scores, ranked

– **Doc A** Employees receive 18 days of annual paid leave. –

– **Doc B** Remote work is permitted up to three days per week with manager approval. –

– **Doc C** Expense claims must be submitted within 30 days of the transaction. –

– **Doc D** The IT helpdesk operates from 9 AM to 6 PM on weekdays. –

### 4 · Retrieved context (Top-2)

–

▾ How this context would be sent to an LLM

–

On Day 2/3 a real LLM API receives a prompt like this. Today the answer on the right is a scripted template.

### 5 · Simulated grounded answer

—

🚫 **TRY "WHAT IS THE COMPANY'S MATERNITY LEAVE POLICY?"**

Document A scores **high** because it is about *leave* — yet it says nothing about maternity leave. **High similarity is not proof that the answer is in the text.** A responsible system must check the evidence and answer "Insufficient information in the available documents." rather than inventing a policy. Real systems combine an instruction to the LLM ("answer only from the context, otherwise say you don't know"), validated score thresholds, re-ranking and evaluation.

---

➡️ **TRANSITION TO THE NEXT TOPIC**

*"The simulation used hand-made features. Real systems use chunks and learned embeddings — let's look at each concept properly."*

05

MODULE 5 · CORE CONCEPTS

# Documents, Chunks, Embeddings and Vector Search

⏱ 20 min

## 5.1 Documents and text extraction

Embedding models and LLMs work on **text**. Every PDF, Word file, HTML page or Markdown note must first be converted into clean, machine-processable text. This is unglamorous work — and it often decides retrieval quality.

### 📕 PDF

Designed for printing, not reading by machines. Watch for headers/footers on every page, multi-column layouts, tables, hyphenated line breaks and scanned pages that need OCR.

### 📝 TXT

Easiest: already text. Still check encoding (UTF-8) and stray whitespace.

### # Markdown / HTML

Has useful structure (headings, lists). Strip markup and navigation but keep headings — they make good chunk boundaries and metadata.

**Before and after text extraction & cleaning**

ILLUSTRATIVE EXAMPLE

**Raw text from a PDF page**

```
ACME Corp — Employee Handbook
Page 4 of 52

3.2   Annual  Leave
Employees receive 18 days of an-
nual paid leave per calendar
year.   Unused days may be car-
ried over (max. 5).

CONFIDENTIAL — Internal use only
4
```

→

**Cleaned text ready for chunking**

```
ACME Corp — Employee Handbook
Page 4 of 52

3.2   Annual  Leave
Employees receive 18 days of an-
nual paid leave per calendar
year.   Unused days may be car-
ried over (max. 5).

CONFIDENTIAL — Internal use only
4
```

**GARBAGE IN, GARBAGE RETRIEVED**

If extraction splits “an-
nual” into two tokens, merges two table columns, or repeats a page header in every chunk, the embeddings describe noise. Retrieval then misses the right passage — and the LLM never gets a chance to answer correctly.

## 5.2 Chunking

A **chunk** is a small passage of a document that is embedded and retrieved as one unit.

### Why not embed a whole document?

One vector for a 40-page handbook blends leave, expenses, IT and security into a single “average meaning”. It matches everything a little and nothing well. And you could not fit many whole documents into the prompt.

### What chunks give you

Focused vectors that match specific questions, small passages that fit in the prompt, and precise source citations (file, page, section).

| SETTING | MEANING | TRADE-OFF |
|---|---|---|
| **Chunk size** | Maximum length of a chunk (characters, words or tokens) | Too small → fragments without enough context (“18 days” without “annual leave”). Too large → mixed topics, blurry vectors, more prompt tokens. |
| **Chunk overlap** | How much text consecutive chunks share | Reduces the chance that a fact is cut in half at a boundary; costs duplicate storage and embedding work. |
| **Fixed-size** | Cut every N characters/tokens | Simple and predictable; may split sentences and words. |
| **Sentence-based** | Group whole sentences up to a size limit | Keeps sentences intact; chunk lengths vary. |

**Chunking simulator** CHARACTER-BASED FOR SIMPLICITY

Input text (edit it to experiment)

```
RAG combines retrieval with generation. Documents are converted into chunks.
Each chunk is represented by an embedding. Relevant chunks are retrieved for a
user question. An LLM uses the retrieved context to generate an answer.
```

| CHUNKS | AVG LENGTH | TEXT LENGTH | DUPLICATED CHARS |
|---|---|---|---|
| 4 | 68 | 227 | 45 |

RAG combines retrieval with generation. Documents are converted into chunks. Each chunk is represented by an embedding. Relevant chunks are retrieved for a user question. An LLM uses the retrieved context to generate an answer.

Hatched, outlined text in the full view = overlap shared by two chunks. In the cards, the highlighted start of a chunk is the part repeated from the previous chunk. This demo splits by **characters**; production systems usually use **token-aware** splitting (matching the embedding model's tokenizer limit) or **structure-aware** splitting (headings, paragraphs, tables).

## 5.3 Embeddings

An **embedding** is a list of numbers (a vector) produced by a neural network to represent the *meaning* of a piece of text. Embedding models are trained so that texts with similar meaning end up with vectors that point in similar directions.

### Why they help search

Keyword search needs shared words. “How many annual leave days are available?” and “Employees receive 18 days of annual paid leave.” share only some words, and a question like “How much holiday do I get?” shares none — yet the meaning matches. Embeddings capture that relationship.

### Model-dependent

Different models produce different vectors, dimensions (e.g. 384 for `all-MiniLM-L6-v2`, 768 or more for others) and score ranges. Vectors from two different models cannot be compared. Always use the **same model** for documents and queries.

**Conceptual picture of two embeddings** ILLUSTRATIVE — NOT REAL MODEL OUTPUT

Each coloured cell stands for one dimension (real models have hundreds). Similar meaning → similar pattern. Pick a second sentence to compare with the question.

“How many annual leave days are available?”



“Employees receive 18 days of annual paid leave.”



ILLUSTRATIVE COSINE

**0.68**

Same topic, different wording → very similar pattern → high score. (16 made-up dimensions; orange = positive, blue = negative.)

🔑 **SHARED MEANING, DIFFERENT WORDS**

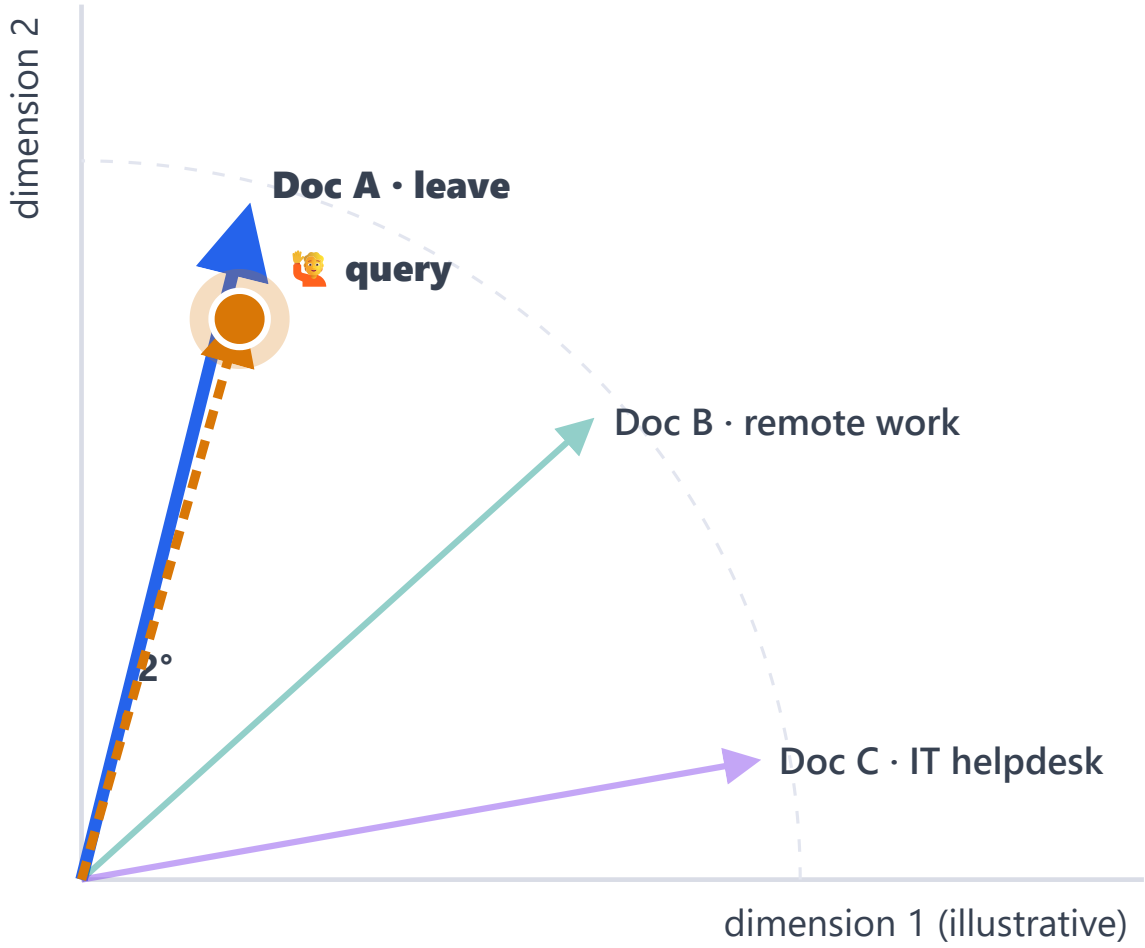
“annual leave days are available” ↔ “receive 18 days of annual paid leave”: both are about *yearly time off* for *employees* measured in *days*. A trained model has seen millions of sentences where “leave”, “vacation”, “time off” and “holiday” appear in similar contexts, so it maps them close together.

## 5.4 Vector search and similarity

**Cosine similarity** measures the angle between two vectors: same direction → 1, perpendicular (unrelated) → 0, opposite → −1. Length does not matter, only direction. If vectors are normalised to length 1, cosine similarity is simply the **dot product**.

**Drag the 🙋 query point — watch the angles and scores**

2-D ILLUSTRATION — REAL EMBEDDINGS HAVE HUNDREDS OF DIMENSIONS




| | | |
|---|---|---|
| 🥇 Doc A · leave | | 1.00 |
| Doc B · remote work | | 0.85 |
| Doc C · IT helpdesk | | 0.43 |

Cosine depends only on the **angle** (2° to the closest document), not the arrow length.

Keyboard: focus the plot and use the arrow keys to move the query.

### Similarity ≠ probability

A cosine of 0.82 does *not* mean "82% likely correct". Scores are relative to a model, a dataset and a metric. One model may give related pairs 0.4–0.6, another 0.7–0.9. Compare scores **within** one setup; choose thresholds only after validating on real questions.

### Why the top-1 chunk is not always enough or right

The answer may be spread over two chunks; a similar-looking chunk (old policy, different country) may outrank the right one; or no chunk may contain the answer at all. Top-K > 1 improves the chance the evidence is included, at the cost of more noise and tokens.

cosine similarity — the whole idea in NumPy — PYTHON

```python
import numpy as np

a = np.array([0.9, 0.3])          # query (illustrative 2-D vector)
b = np.array([0.8, 0.5])          # document

cosine = a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
print(round(float(cosine), 3))    # 0.974 → almost the same direction
```

**CHECK 4**

## What is chunk overlap mainly for?

**A. Reducing the chance that information is split across a chunk boundary and lost**

B. Making the vector store smaller

C. Encrypting the chunks

D. Increasing the embedding dimension

---

➡️ **TRANSITION TO THE NEXT TOPIC**

*"Concepts are in place. Time to build a real retrieval pipeline in Python with a real embedding model."*

06

MODULE 6 · LIVE CODING — THE CRITICAL BLOCK

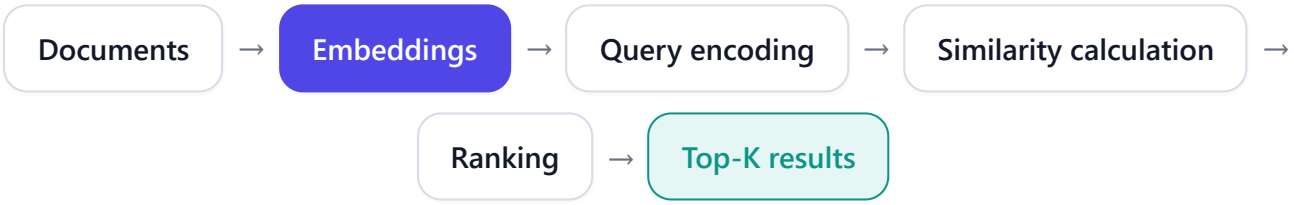
# Live Python Coding Walkthrough: First Retrieval Pipeline

⏱ 35–40 min

We now replace the illustrations with a **real embedding model** — `sentence-transformers/all-MiniLM-L6-v2` — running on your machine. Vectors are kept in a NumPy array in memory: no database, no API key, no framework.

📍 **RETRIEVAL ONLY — NOT YET A COMPLETE RAG APPLICATION**

This program **finds** the most relevant passages and shows their scores and sources. It does **not** generate a natural-language answer, because it does not call an LLM. That generation step is added in a later session; Step 8 previews it.




## Step 1 — Environment setup

Create a fresh folder `rag-day1/`, open a terminal in it and create a virtual environment.

PowerShell — inside rag-day1\ BASH

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install sentence-transformers numpy
```

terminal — inside rag-day1/ BASH

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install sentence-transformers numpy
```

`python -m venv .venv` Isolated environment for today's packages (same habit as Day 1 FastAPI).

| | |
|---|---|
| `Activate.ps1` | Your prompt shows `(.venv)` when active. All `pip` and `python` commands now use this environment. |
| `sentence-transformers` | Loads pre-trained embedding models; installs PyTorch and Hugging Face `transformers` as dependencies (a few hundred MB — install before class). |
| `numpy` | Matrix maths for similarity scores. |

🌐 **FIRST RUN NEEDS INTERNET**

The first time the code loads `all-MiniLM-L6-v2` (about 90 MB) it is downloaded from the Hugging Face Hub and cached (on Windows usually under `%USERPROFILE%\.cache\huggingface`). Later runs work offline from the cache.

▾ 🔧 Troubleshooting the setup

| SYMPTOM | CAUSE | FIX |
|---|---|---|
| `Activate.ps1 cannot be loaded because running scripts is disabled` | PowerShell execution policy | `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, reopen the terminal; or use `.venv\Scripts\activate.bat` in cmd.exe |
| `pip` installs to the wrong Python | Environment not active | Check `where python` (Windows) / `which python`: it must point into `.venv`. Use `python -m pip install …` |
| No matching distribution for `torch` | Python version too new/old or 32-bit Python | Use 64-bit Python 3.10–3.12 (3.13 works with recent PyTorch). Check with `python --version` |
| Very slow / huge download | PyTorch with GPU (CUDA) libraries on Linux | CPU is enough today: `pip install torch --index-url https://download.pytorch.org/whl/cpu` first, then `sentence-transformers` |
| `OSError: … couldn't connect to huggingface.co` | No internet / corporate proxy on first run | Connect once, set `HTTPS_PROXY`, or copy a pre-downloaded model folder and pass its path to `SentenceTransformer("path/to/model")` |
| `ModuleNotFoundError: sentence_transformers` | Running a different interpreter (e.g. IDE) | Select `.venv` as the interpreter in VS Code / PyCharm |

Quick check that everything works:

sanity check BASH

```
python -c "from sentence_transformers import SentenceTransformer; m = SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2'); print(m.encode(['hello']).shape)"
```

Expected: `(1, 384)`, possibly after download progress bars.

## Step 2 — Define the knowledge base

Create `rag_retrieval.py`. Real systems produce chunks with an ingestion pipeline; today we write six small, already-chunked policy passages by hand. Each has an **ID**, the **text**, its **source** document and optional **metadata**.

rag_retrieval.py — imports and knowledge base PYTHON

```python
"""Day 1 - RAG Fundamentals: first retrieval pipeline (no LLM, no API key).

Documents -> Embeddings -> Query encoding -> Cosine similarity -> Ranking -> Top-K

Run:
    python rag_retrieval.py
"""
from __future__ import annotations

import sys

import numpy as np

try:
    from sentence_transformers import SentenceTransformer
except ImportError:  # clear message instead of a long traceback
    sys.exit("sentence-transformers is not installed. Run: pip install sentence-transformers numpy")

MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"

# ---------------------------------------------------------------------------
# Step 2 - Knowledge base: small, pre-chunked company-policy passages
# ---------------------------------------------------------------------------
KNOWLEDGE_BASE: list[dict] = [
    {
        "id": "HR-001",
        "text": "Employees receive 18 days of annual paid leave per calendar year.",
        "source": "hr_leave_policy.pdf",
        "metadata": {"department": "HR", "updated": "2026-01-15"},
    },
    {
        "id": "HR-002",
        "text": "Remote work is permitted up to three days per week with manager approval.",
        "source": "remote_work_policy.md",
        "metadata": {"department": "HR", "updated": "2026-03-02"},
    },
    {
        "id": "FIN-001",
        "text": "Expense claims must be submitted within 30 days of the transaction date.",
        "source": "finance_expenses.pdf",
        "metadata": {"department": "Finance", "updated": "2025-11-20"},
    },
```

```
    {
        "id": "IT-001",
        "text": "The IT helpdesk operates from 9 AM to 6 PM on weekdays.",
        "source": "it_support_guide.txt",
        "metadata": {"department": "IT", "updated": "2026-02-10"},
    },
    {
        "id": "IT-002",
        "text": "Employees must change their network password every 90 days.",
        "source": "it_security_policy.pdf",
        "metadata": {"department": "IT", "updated": "2026-04-01"},
    },
    {
        "id": "FAC-001",
        "text": "Visitors must sign in at reception and wear a visitor badge at all times.",
        "source": "office_facilities.md",
        "metadata": {"department": "Facilities", "updated": "2025-09-05"},
    },
]
```

| | |
|---|---|
| `from __future__ import annotations` | Allows modern type hints like `list[dict]` on older Python versions. |
| `try: … except ImportError` | Prints a clear install instruction instead of a long traceback. |
| `MODEL_NAME` | One constant, used for indexing *and* querying — guarantees the same model on both sides. |
| `"id"` | Stable chunk identifier: lets you trace results, update or delete chunks later. |
| `"source"` | Where the text came from — needed for citations in the final answer. |
| `"metadata"` | Extra facts for filtering later (e.g. only HR documents, only documents updated this year). |

## Step 3 — Generate embeddings

rag_retrieval.py — load the model and index the chunks PYTHON

```python
def load_model(name: str = MODEL_NAME) -> SentenceTransformer:
    try:
        return SentenceTransformer(name)
    except Exception as exc:  # network error, wrong name, corrupted cache ...
        sys.exit(
            f"Could not load embedding model '{name}': {exc}\n"
            "The first run downloads the model and needs internet access."
        )


def build_index(model: SentenceTransformer, chunks: list[dict]) -> np.ndarray:
    if not chunks:
        raise ValueError("The knowledge base is empty - add at least one chunk.")
    texts = [chunk["text"] for chunk in chunks]
    # normalize_embeddings=True -> every vector has length 1,
```

```
    # so a dot product equals cosine similarity.
    embeddings = model.encode(texts, normalize_embeddings=True, convert_to_numpy=True)
    return embeddings.astype(np.float32)  # shape: (num_chunks, 384)
```

| | |
|---|---|
| `SentenceTransformer(name)` | Downloads (first time) and loads the model. Loading takes a few seconds — do it **once**, not per query. |
| `sys.exit( ... )` | Fails with a helpful message for network or name errors. |
| `texts = [ ... ]` | Input: a list of 6 strings. |
| `model.encode(texts, …)` | Runs the neural network on every chunk. Output: a NumPy array of shape `(6, 384)` — one row per chunk, 384 numbers per row. |
| `normalize_embeddings=True` | Scales every vector to length 1. Then *dot product = cosine similarity*, which keeps Step 4 simple and fast. |
| `astype(np.float32)` | Compact 32-bit floats — what vector databases typically store. |

**Input**
6 chunk texts
`list[str], len 6`

**Output**
Embedding matrix
`(6, 384) float32`

**Property**
Each row has length 1
`np.linalg.norm(row) ≈ 1.0`

## Steps 4 and 5 — Similarity search inside a reusable `retrieve()` function

Click any highlighted line (or use **Next line**) for its explanation.

```
1  def retrieve(
2      query: str,
3      model: SentenceTransformer,
4      doc_embeddings: np.ndarray,
5      chunks: list[dict],
6      top_k: int = 3,
7  ) -> list[dict]:
8      """Return the top_k most similar chunks with their scores and metadata."""
9      if not query or not query.strip():
10         raise ValueError("Query must be a non-empty string.")
11     if top_k < 1:
12         raise ValueError("top_k must be at least 1.")
13     top_k = min(top_k, len(chunks))
14
15     # Same model + same normalisation as indexing → vectors are comparable.
16     query_vec = model.encode([query], normalize_embeddings=True, convert_to_numpy=True)
17
18     # (num_chunks, 384) @ (384,) → (num_chunks,) cosine similarities in [-1, 1]
```

```
    scores = doc_embeddings @ query_vec

    # Indices of the highest scores, best first.
    ranked = np.argsort(scores)[::-1][:top_k]

    results = []
    for rank, idx in enumerate(ranked, start=1):
        chunk = chunks[idx]
        results.append(
            {
                "rank": rank,
                "id": chunk["id"],
                "score": float(scores[idx]),
                "text": chunk["text"],
                "source": chunk["source"],
                "metadata": chunk["metadata"],
            }
        )
    return results
```

**LINE-BY-LINE WALKTHROUGH**

Select a line on the left.

**THE ONE LINE THAT DOES THE SEARCH**

`scores = doc_embeddings @ query_vec` multiplies a `(6, 384)` matrix by a `(384,)` vector → `(6,)` : one cosine similarity per chunk. Because both sides are normalised, each value is in [−1, 1]. With a million chunks this brute-force approach gets slow, which is why vector databases use *approximate nearest-neighbour* indexes — a later topic.

## Step 6 — Run example queries

rag_retrieval.py — printing and the main block PYTHON

```python
def print_results(query: str, results: list[dict]) -> None:
    print(f"\nQuery: {query}")
    for r in results:
        print(f"  #{r['rank']}  {r['id']:<8} score={r['score']:.3f}  source={r['source']}")
        print(f"       {r['text']}")


# ─────────────────────────────────────────────────────────────
# Step 6 — Run example queries
# ─────────────────────────────────────────────────────────────
if __name__ == "__main__":
    model = load_model()
```

```
    doc_embeddings = build_index(model, KNOWLEDGE_BASE)
    print(f"Indexed {len(KNOWLEDGE_BASE)} chunks → embedding matrix shape {doc_embeddings.shape}")

    queries = [
        "How many annual leave days do employees receive?",
        "Can I work from home?",
        "What is the deadline for submitting expense claims?",
        "When can I contact IT support?",
        "What is the company's maternity leave policy?",  # NOT in the knowledge base
    ]
    for q in queries:
        print_results(q, retrieve(q, model, doc_embeddings, KNOWLEDGE_BASE, top_k=3))

    # Step 8 preview: what would be sent to an LLM for the first question
    first = retrieve(queries[0], model, doc_embeddings, KNOWLEDGE_BASE, top_k=2)
    print("\n--- Prompt preview (not sent anywhere today) ---")
    print(build_prompt(queries[0], first))
```

run it BASH

```
python rag_retrieval.py
```

**EXPECTED OUTPUT PATTERN** The format is exactly what the script prints; the **top-ranked chunk** shown for each question is the expected result. Scores are deliberately written as `0.xx` and lower ranks as `…`: exact values depend on the model revision, library versions and hardware, so read the real numbers from *your* run during the live demo rather than from this page.

```
Indexed 6 chunks → embedding matrix shape (6, 384)

Query: How many annual leave days do employees receive?
  #1  HR-001   score=0.xx  source=hr_leave_policy.pdf
       Employees receive 18 days of annual paid leave per calendar year.
  #2  …        score=0.xx  (a less related chunk)
  #3  …        score=0.xx  (a less related chunk)

Query: Can I work from home?
  #1  HR-002   score=0.xx  source=remote_work_policy.md
       Remote work is permitted up to three days per week with manager approval.
  #2  …

Query: What is the deadline for submitting expense claims?
  #1  FIN-001  score=0.xx  source=finance_expenses.pdf
       Expense claims must be submitted within 30 days of the transaction date.
  #2  …

Query: When can I contact IT support?
  #1  IT-001   score=0.xx  source=it_support_guide.txt
       The IT helpdesk operates from 9 AM to 6 PM on weekdays.
  #2  …
```

```
Query: What is the company's maternity leave policy?
  #1  HR-001   score=0.xx  source=hr_leave_policy.pdf      ← related topic, NOT the ans
       Employees receive 18 days of annual paid leave per calendar year.
  #2  …
  #3  …

--- Prompt preview (not sent anywhere today) ---
You are a company policy assistant.
Answer the question using ONLY the context below.
If the context does not contain the answer, reply exactly:
"Insufficient information in the available documents."
Cite the source of every fact in square brackets.

Context:
[hr_leave_policy.pdf] Employees receive 18 days of annual paid leave per calendar year.
[<second-ranked source>] <second-ranked chunk text>

Question: How many annual leave days do employees receive?
Answer:
```

### ✅ What worked

Each supported question should rank the right chunk first — including “Can I work from home?”, which shares almost no words with “Remote work is permitted…”. That is semantic search.

### ⚠️ The maternity question

The knowledge base has no maternity policy, yet retrieval still returns three chunks, the leave policy first. Ranking **always** returns the K nearest chunks — it never says “nothing found”. Its top score is often somewhat lower than for supported questions, but there is **no universal threshold**: a cut-off must be chosen and validated on labelled questions for your model and data, and even then an LLM instruction and evidence check are needed.

## Step 7 — The retrieval pipeline, traced

**Annotated execution trace for "How many annual leave days do employees receive?"**

SHAPES EXACT · NUMBERS ILLUSTRATIVE

| | | |
|---|---|---|
| 1 | Documents | `KNOWLEDGE_BASE: 6 dicts`<br>`'Employees receive 18 days of annual paid leave per calendar year.' …` |
| 2 | Embeddings | `doc_embeddings = build_index(model, KNOWLEDGE_BASE)`<br>`→ shape (6, 384) float32, each row has norm 1.0 (indexing: done once)` |
| 3 | Query encoding | `query_vec = model.encode(['How many annual leave days do employees receive?'], normalize_embeddings=True)[0]`<br>`→ shape (384,)` |
| 4 | Similarity calculation | `scores = doc_embeddings @ query_vec`<br>`→ shape (6,) e.g. [0.7x, 0.1x, 0.1x, 0.0x, 0.1x, 0.0x] (illustrative values)` |
| 5 | Ranking | `np.argsort(scores)[::-1]`<br>`→ indices best-first, e.g. [0, 2, 1, …] (index 0 = HR-001)` |
| 6 | Top-K results | `[:3] → 3 dicts with rank, id, score, text, source, metadata`<br>`#1 HR-001 hr_leave_policy.pdf 'Employees receive 18 days…'` |

## Step 8 — How this becomes RAG PREVIEW

Retrieved chunks → Context assembly → Prompt template → LLM API (Day 2/3) → Generated answer + citations

The **augment** step is plain string building. The template tells the model to use only the context, to abstain when the answer is missing, and to cite sources. It is already in the script and printed at the end — but **nothing is sent to any API today**.

rag_retrieval.py — prompt template (augment step) PYTHON

```
PROMPT_TEMPLATE = """You are a company policy assistant.
Answer the question using ONLY the context below.
If the context does not contain the answer, reply exactly:
"Insufficient information in the available documents."
```

```
Cite the source of every fact in square brackets.

Context:
{context}

Question: {question}
Answer:"""


def build_prompt(query: str, results: list[dict]) -> str:
    context = "\n".join(f"[{r['source']}] {r['text']}" for r in results)
    return PROMPT_TEMPLATE.format(context=context, question=query)
```

SOON

**NEXT SESSION**

We will send this prompt to an LLM API (with the key read from an environment variable, never hard-coded), handle the “insufficient information” case, return citations, and wrap it in a FastAPI endpoint.

## Complete `rag_retrieval.py` (copy-ready)

rag_retrieval.py — complete file PYTHON

```python
"""Day 1 - RAG Fundamentals: first retrieval pipeline (no LLM, no API key).

Documents → Embeddings → Query encoding → Cosine similarity → Ranking → Top-K

Run:
    python rag_retrieval.py
"""
from __future__ import annotations

import sys

import numpy as np

try:
    from sentence_transformers import SentenceTransformer
except ImportError:  # clear message instead of a long traceback
    sys.exit("sentence-transformers is not installed. Run: pip install sentence-transformers numpy")

MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"

# ---------------------------------------------------------------------------
# Step 2 - Knowledge base: small, pre-chunked company-policy passages
# ---------------------------------------------------------------------------
KNOWLEDGE_BASE: list[dict] = [
    {
        "id": "HR-001",
        "text": "Employees receive 18 days of annual paid leave per calendar year.",
        "source": "hr_leave_policy.pdf",
        "metadata": {"department": "HR", "updated": "2026-01-15"},
    },
    {
        "id": "HR-002",
        "text": "Remote work is permitted up to three days per week with manager approval.",
        "source": "remote_work_policy.md",
        "metadata": {"department": "HR", "updated": "2026-03-02"},
    },
    {
        "id": "FIN-001",
        "text": "Expense claims must be submitted within 30 days of the transaction date.",
        "source": "finance_expenses.pdf",
        "metadata": {"department": "Finance", "updated": "2025-11-20"},
    },
    {
        "id": "IT-001",
        "text": "The IT helpdesk operates from 9 AM to 6 PM on weekdays.",
        "source": "it_support_guide.txt",
        "metadata": {"department": "IT", "updated": "2026-02-10"},
    },
    {
```

```
        "id": "IT-002",
        "text": "Employees must change their network password every 90 days.",
        "source": "it_security_policy.pdf",
        "metadata": {"department": "IT", "updated": "2026-04-01"},
    },
    {
        "id": "FAC-001",
        "text": "Visitors must sign in at reception and wear a visitor badge at all times.",
        "source": "office_facilities.md",
        "metadata": {"department": "Facilities", "updated": "2025-09-05"},
    },
]


# ---------------------------------------------------------------------------
# Step 3 - Load the embedding model and embed every chunk (indexing)
# ---------------------------------------------------------------------------
def load_model(name: str = MODEL_NAME) -> SentenceTransformer:
    try:
        return SentenceTransformer(name)
    except Exception as exc:  # network error, wrong name, corrupted cache ...
        sys.exit(
            f"Could not load embedding model '{name}': {exc}\n"
            "The first run downloads the model and needs internet access."
        )


def build_index(model: SentenceTransformer, chunks: list[dict]) -> np.ndarray:
    if not chunks:
        raise ValueError("The knowledge base is empty - add at least one chunk.")
    texts = [chunk["text"] for chunk in chunks]
    # normalize_embeddings=True -> every vector has length 1,
    # so a dot product equals cosine similarity.
    embeddings = model.encode(texts, normalize_embeddings=True, convert_to_numpy=True)
    return embeddings.astype(np.float32)  # shape: (num_chunks, 384)


# ---------------------------------------------------------------------------
# Steps 4 and 5 - Similarity search wrapped in a reusable retrieve() function
# ---------------------------------------------------------------------------
def retrieve(
    query: str,
    model: SentenceTransformer,
    doc_embeddings: np.ndarray,
    chunks: list[dict],
    top_k: int = 3,
) -> list[dict]:
    """Return the top_k most similar chunks with their scores and metadata."""
    if not query or not query.strip():
        raise ValueError("Query must be a non-empty string.")
    if top_k < 1:
        raise ValueError("top_k must be at least 1.")
    top_k = min(top_k, len(chunks))

    # Same model + same normalisation as indexing -> vectors are comparable.
    query_vec = model.encode([query], normalize_embeddings=True, convert_to_numpy=True)[0]
```

```
    # (num_chunks, 384) @ (384,) → (num_chunks,) cosine similarities in [-1, 1]
    scores = doc_embeddings @ query_vec

    # Indices of the highest scores, best first.
    ranked = np.argsort(scores)[::-1][:top_k]

    results = []
    for rank, idx in enumerate(ranked, start=1):
        chunk = chunks[idx]
        results.append(
            {
                "rank": rank,
                "id": chunk["id"],
                "score": float(scores[idx]),
                "text": chunk["text"],
                "source": chunk["source"],
                "metadata": chunk["metadata"],
            }
        )
    return results


# ─────────────────────────────────────────────────────────────────────
# Step 8 - Preview of the "augment" step: build the prompt an LLM would receive
# (no API call today - Day 2/3 sends this to an LLM)
# ─────────────────────────────────────────────────────────────────────
PROMPT_TEMPLATE = """You are a company policy assistant.
Answer the question using ONLY the context below.
If the context does not contain the answer, reply exactly:
"Insufficient information in the available documents."
Cite the source of every fact in square brackets.

Context:
{context}

Question: {question}
Answer:"""


def build_prompt(query: str, results: list[dict]) → str:
    context = "\n".join(f"[{r['source']}] {r['text']}" for r in results)
    return PROMPT_TEMPLATE.format(context=context, question=query)


def print_results(query: str, results: list[dict]) → None:
    print(f"\nQuery: {query}")
    for r in results:
        print(f"  #{r['rank']}  {r['id']:<8} score={r['score']:.3f}  source={r['source']}")
        print(f"       {r['text']}")


# ─────────────────────────────────────────────────────────────────────
# Step 6 - Run example queries
# ─────────────────────────────────────────────────────────────────────
if __name__ == "__main__":
```

```
model = load_model()
doc_embeddings = build_index(model, KNOWLEDGE_BASE)
print(f"Indexed {len(KNOWLEDGE_BASE)} chunks → embedding matrix shape {doc_embeddings.shape}")

queries = [
    "How many annual leave days do employees receive?",
    "Can I work from home?",
    "What is the deadline for submitting expense claims?",
    "When can I contact IT support?",
    "What is the company's maternity leave policy?",  # NOT in the knowledge base
]
for q in queries:
    print_results(q, retrieve(q, model, doc_embeddings, KNOWLEDGE_BASE, top_k=3))

# Step 8 preview: what would be sent to an LLM for the first question
first = retrieve(queries[0], model, doc_embeddings, KNOWLEDGE_BASE, top_k=2)
print("\n--- Prompt preview (not sent anywhere today) ---")
print(build_prompt(queries[0], first))
```

**CHECK 5**

## Why must queries be encoded with the same model used for the documents?

A. It is faster

**B. Different models produce incompatible vector spaces, so scores would be meaningless**

C. The licence requires it

D. The vector store can only hold one model

---

➡️ **TRANSITION TO THE NEXT TOPIC**

*"The pipeline works. Now practise: add documents, change Top-K, and probe questions the knowledge base cannot answer."*

07

MODULE 7 · YOUR TURN

# Guided Hands-On Exercises

⏱ 10–15 min

All exercises build on `rag_retrieval.py` from Module 6. Try first; open the solution afterwards.

### 🛠️ Exercise 1 — Add two documents and retrieve them

Beginner · 5 min

## Objective

Show that new knowledge becomes searchable just by indexing it — no model training.

## Instructions

1. Append the two entries below to `KNOWLEDGE_BASE`.
2. Re-run the script (this re-builds the index).
3. Ask two natural-language questions that use *different words* from the documents.

add to KNOWLEDGE_BASE — PYTHON

```python
{
    "id": "HR-003",
    "text": "New employees complete a probation period of six months.",
    "source": "hr_onboarding.pdf",
    "metadata": {"department": "HR", "updated": "2026-05-12"},
},
{
    "id": "FIN-002",
    "text": "Business-class flights are only allowed for journeys longer than eight hours.",
    "source": "finance_travel_policy.pdf",
    "metadata": {"department": "Finance", "updated": "2026-06-01"},
},
```

test queries — add after the existing loop — PYTHON

```python
for q in ["How long is the trial period for new starters?",
          "Can I fly business class to New York?"]:
    print_results(q, retrieve(q, model, doc_embeddings, KNOWLEDGE_BASE, top_k=2))
```

## Expected behaviour

`HR-003` should rank first for the trial-period question and `FIN-002` first for the flight question, although “trial period”, “new starters” and “fly” do not appear literally in the documents. The index shape becomes `(8, 384)`.

▾ Solution & explanation

Updating knowledge in RAG = re-running ingestion for changed documents. Point out that `build_index` must run *after* the list changes; if consultants add entries below the `build_index` call, the new chunks are silently missing — a realistic production bug (index out of date). In a real system you would embed and insert only the new chunks instead of rebuilding everything.

### 🛠️ Exercise 2 — Change Top-K from 1 to 3

Beginner · 4 min

## Objective

See the trade-off between missing evidence (K too small) and adding noise (K too large).

## Instructions

1. For the question `"What are the rules for remote work and expense claims?"` call `retrieve` with `top_k=1`, then `top_k=3`.
2. Also try it in the Module 4 simulator with the Top-K slider.

compare K PYTHON

```python
q = "What are the rules for remote work and expense claims?"
for k in (1, 3):
    print(f"\n═══ top_k={k} ═══")
    print_results(q, retrieve(q, model, doc_embeddings, KNOWLEDGE_BASE, top_k=k))
```

Open the simulator with a remote-work question ↗

## Expected behaviour

With K = 1 only one of the two needed policies is returned, so an LLM could answer only half the question. With K = 3 both relevant chunks are very likely present, plus one less relevant chunk.

▾ Solution & explanation

Multi-part questions need larger K (or query decomposition — later topic). Larger K costs prompt tokens and can distract the model. Typical starting values are 3–5, then tuned with evaluation on real questions.

### 🛠️ Exercise 3 — Ask what the knowledge base cannot answer

Intermediate · 5 min

## Objective

Understand why retrieval always returns something, and why applications need evidence checks and an abstention strategy.

## Instructions

1. Ask: `"How many sick days do employees get?"` and `"What is the parking policy?"`
2. Note the top chunk and score for each, and compare with the scores of supported questions.
3. Write down how you would decide, in production, that the system should answer "Insufficient information in the available documents."

probe unsupported questions — PYTHON

```python
for q in ["How many sick days do employees get?", "What is the parking policy?"]:
    results = retrieve(q, model, doc_embeddings, KNOWLEDGE_BASE, top_k=3)
    print_results(q, results)
    print(f"  top score: {results[0]['score']:.3f}  ← compare with supported questions")
```

Try the maternity question in the simulator ↗

## Expected behaviour

The sick-days question very likely returns the *annual leave* chunk first — superficially related (leave, days, employees) but not the answer. The parking question likely returns the visitor/reception chunk. Scores may be lower than for supported questions, but not by a fixed, reliable margin.

▾ Solution & explanation

- **Why it happens:** `argsort` ranks every chunk; nearest neighbours exist even when nothing is relevant.
- **Abstention strategies (combine them):** an instruction in the prompt to answer only from context and otherwise reply with the fixed insufficient-information message; a score threshold *chosen from a labelled validation set* for this model and corpus; a re-ranker or an LLM-based relevance check on the retrieved chunks; requiring citations and checking they support the answer; logging low-confidence cases for review.
- **Why no universal threshold:** score distributions differ between models, languages, chunk sizes and domains.

➡️ **TRANSITION TO THE NEXT TOPIC**

*"Exercises done. Let's check understanding with a scored quiz and practise consultant interview questions."*

08

MODULE 8 · ASSESS

# Knowledge Check and Consultant Interview Questions

⏱ 10 min

Ten questions. Your **first answer** counts and the explanation appears immediately.

📝 Q1 · WHAT RAG IS

### Which description of RAG is most accurate?

A. Training an LLM on your documents

B. A vector database product

**C. Retrieving relevant passages at query time and adding them to the LLM prompt**

D. A prompt that tells the LLM to be accurate

📝 Q2 · WHY RETRIEVAL

### Why is retrieval needed for questions about internal company policies?

**A. The LLM's weights don't contain the private, current policy text**

B. LLMs cannot read English policies

C. Retrieval makes the model larger

D. It is required by FastAPI

📝 **Q3 · RAG VS FINE-TUNING**

### A model must always produce answers in a specialised house format. The facts are already in the prompt. What fits best if prompting alone is not reliable enough?

A. Add more documents to the vector store

B. Increase Top-K

C. Use a bigger chunk size

**D. Fine-tuning on examples of the required format**

📝 **Q4 · CHUNKING**

### What is the main risk of very large chunks?

A. Facts get cut in half

**B. Each vector blends many topics, so retrieval is less precise and prompts get longer**

C. The embedding model refuses them

D. Overlap stops working

📝 **Q5 · EMBEDDINGS**

### Which statement about embeddings is correct?

A. An embedding is a compressed, readable copy of the text

B. All embedding models produce 384 dimensions

**C. It is a vector where similar meanings tend to point in similar directions**

D. Embeddings are probabilities

📝 **Q6 · VECTOR SEARCH**

### In the Python code, why can we use `doc_embeddings @ query_vec` as cosine similarity?

**A. Both were encoded with `normalize_embeddings=True`, so the dot product equals cosine**

B. NumPy's `@` always computes cosine

C. Because Top-K is 3

D. Because the vectors are float32

📝 **Q7 · TOP-K**

### What happens when you increase Top-K?

A. Scores become more accurate

B. The model is retrained

C. Fewer chunks are sent

**D. More of the ranked chunks are passed on — more chance to include evidence, more noise and tokens**

📝 **Q8 · INGESTION VS QUERY**

### Which steps belong to the *query* pipeline?

A. Text extraction, cleaning, chunking

**B. Query embedding, similarity search, Top-K, context assembly, generation**

C. Embedding all documents

D. Writing vectors into the vector store

📝 Q9 · SIMILARITY ≠ CORRECTNESS

### The top chunk has a high similarity score. What can you conclude?

A. The answer is certainly in that chunk

B. There is a matching probability of being correct

**C. Only that it is the closest chunk in this vector space — it may still not contain the answer**

D. The LLM will not hallucinate

📝 Q10 · RETRIEVAL-ONLY

### Why is today's Python program not yet a complete RAG application?

**A. It retrieves context but never calls an LLM to generate an answer from it**

B. It does not use embeddings

C. It has no Docker image

D. It uses NumPy instead of a vector database

YOUR SCORE

**0 / 10**

0 of 10 answered — keep going.

## 🎤 Consultant interview questions

▾ 1. Explain RAG to a client's CTO in two minutes. When would you not use it?

RAG keeps the LLM unchanged and, for each question, searches the client's documents for the most relevant passages, then asks the LLM to answer from those passages with citations. It suits large, private or frequently changing knowledge. I would not use it when the whole knowledge fits comfortably in the prompt and rarely changes (just prompt), when the need is a behaviour or format change (prompting or fine-tuning), or when answers require exact computation over structured data (query the database or call a tool instead).

### 2. Retrieval quality is poor. Where do you look first?

Upstream first: inspect extracted text and chunks for noise (headers, broken tables, hyphenation) and check chunk size/boundaries. Then confirm the same embedding model is used for documents and queries and that the index is up to date. Then look at the model choice for the domain and language, Top-K, metadata filters, hybrid keyword + vector search and re-ranking. Measure with a small labelled set of question → expected chunk pairs (e.g. hit rate @K) instead of eyeballing.

### 3. How do you choose chunk size and overlap?

Start from the document structure and the question types: FAQ entries or policy clauses can be single small chunks; long technical prose needs paragraph-sized chunks with some overlap. Respect the embedding model's token limit. Then evaluate a few configurations on real questions — chunking is an empirical choice with a trade-off between precision (small) and context completeness (large), and overlap trades duplication for fewer split facts.

### 4. Is a cosine score of 0.8 good? Can we use 0.7 as the "found an answer" threshold?

A score is only meaningful relative to the model, metric and corpus — it is not a probability. Some models compress scores into a narrow band. A threshold can help as one signal, but it must be calibrated on labelled examples (answerable vs not) and monitored. It should be combined with prompt instructions to abstain, re-ranking or relevance checks, and citation verification.

### 5. What changes when this prototype goes to production?

A proper ingestion job (loaders, cleaning, chunking, incremental updates and deletions), a vector database with ANN indexing and metadata filtering, access control so users only retrieve documents they may see, an LLM call with prompt templates, citations and abstention, evaluation of retrieval and answers, monitoring of latency/cost/quality, secrets in environment variables or a secret manager, and deployment as a FastAPI service in Docker — topics for the coming days.

---

➡️ **TRANSITION TO THE NEXT TOPIC**

*"Quiz done. Let's recap the day and prepare for Day 2."*

09

MODULE 9 · WRAP-UP

# Recap and Preparation for Day 2

⏱ 5 min

### ❓ The problem

LLMs know only their training data and the prompt. Private, changing documents need to be **retrieved** and put into the prompt.

### 🔀 Two pipelines

**Ingestion** (offline): extract → clean → chunk → embed → store. **Query** (online): embed question → search → Top-K → context → LLM → answer.

### 🧱 Building blocks

**Chunks** are retrievable units; **embeddings** represent meaning as vectors; **vector search** ranks chunks by cosine similarity.

### 🪢 What you built

A working semantic retrieval pipeline: real `all-MiniLM-L6-v2` embeddings, NumPy cosine search, Top-K with scores and sources.

### 🔍 Retrieval ≠ generation

Retrieval finds evidence; it never writes an answer and can return related-but-wrong chunks. Generation needs an LLM, a prompt and an abstention rule.

### 🧭 Still missing

LLM call and prompt construction, citations, abstention, better ingestion and chunking, evaluation, a vector database and deployment.

## SOON Day 2 preview

Better document ingestion → Chunking experiments → Better retrieval →

Prompt construction → **LLM integration** → Grounded answers + citations

Bring today's `rag_retrieval.py` and a working `.venv`. Have an LLM API key from your organisation ready — stored in an environment variable, never in code.

## ✅ Final checklist

Tick what you can do now. **0 / 10**

- [ ] Explain what RAG is and why it is useful
- [ ] Name the limits of a standalone LLM
- [ ] Choose between RAG, fine-tuning and prompting
- [ ] Draw the ingestion and query pipelines
- [ ] Explain chunk size and overlap trade-offs
- [ ] Explain what an embedding is (and is not)

- [ ] Explain cosine similarity and Top-K
- [ ] Run `rag_retrieval.py` with a real embedding model
- [ ] Explain why similarity does not prove an answer is supported
- [ ] Explain retrieval vs generation

---



# Day 1 Complete

You can explain why RAG exists, draw its two pipelines, reason about chunks, embeddings and similarity, and you have a real semantic retrieval pipeline running in Python. Next: turning retrieved context into grounded, cited answers.